



 Latest updates: <https://dl.acm.org/doi/10.1145/1498698.1594230>

RESEARCH-ARTICLE

Cache-, hash-, and space-efficient bloom filters

FELIX PUTZE, Karlsruhe Institute of Technology, Campus South, Karlsruhe, Baden-Wurttemberg, Germany

PETER SANDERS, Karlsruhe Institute of Technology, Campus South, Karlsruhe, Baden-Wurttemberg, Germany

JOHANNES SINGLER, Karlsruhe Institute of Technology, Campus South, Karlsruhe, Baden-Wurttemberg, Germany

Open Access Support provided by:

Karlsruhe Institute of Technology, Campus South

Published: 05 January 2010

Accepted: 01 July 2009

Received: 01 January 2009

[Citation in BibTeX format](#)

Cache-, Hash-, and Space-Efficient Bloom Filters

FELIX PUTZE, PETER SANDERS, and JOHANNES SINGLER
Universität Karlsruhe

A Bloom filter is a very compact data structure that supports approximate membership queries on a set, allowing false positives.

We propose several new variants of Bloom filters and replacements with similar functionality. All of them have a better cache-efficiency and need less hash bits than regular Bloom filters. Some use SIMD functionality, while the others provide an even better space efficiency. As a consequence, we get a more flexible trade-off between false-positive rate, space-efficiency, cache-efficiency, hash-efficiency, and computational effort. We analyze the efficiency of Bloom filters and the proposed replacements in detail, in terms of the false-positive rate, the number of expected cache-misses, and the number of required hash bits. We also describe and experimentally evaluate the performance of highly tuned implementations. For many settings, our alternatives perform better than the methods proposed so far.

Categories and Subject Descriptors: E.2.7 [Data Storage Representations]: Hash-Table Representations; F.2.2 [Analysis of Algorithms and Problem Complexity]: Nonnumerical Algorithms and Problems—*Sorting; searching*; F.2.3 [Analysis of Algorithms and Problem Complexity]: Tradeoffs between Complexity Measures

General Terms: Algorithms, Experimentation, Measurement, Performance, Theory

Additional Key Words and Phrases: Approximate dictionary, data compression

ACM Reference Format:

Putze, F., Sanders, P., and Singler, J. 2008. Cache-, Hash-, and space-efficient bloom filters. *ACM J. Exp. Algor.* 14, Article 4.4 (September 2008), 18 pages.
DOI = 10.1145/1498698.1594230 <http://doi.acm.org/10.1145/1498698.1594230>

1. INTRODUCTION

The term Bloom filter names a data structure that supports membership queries on a set of elements, and it was introduced already in 1970 by Burton Bloom [1970]. It differs from ordinary dictionary data structures, as the result of a membership query might be “true” although the element is not

Authors’ email addresses: {putze,sanders,singler}@ira.uka.de

Partially supported by DFG grant SA 933/3-1.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.
© 2009 ACM 1084-6654/2009/09-ART4.4 \$10.00

DOI 10.1145/1498698.1594230 <http://doi.acm.org/10.1145/1498698.1594230>

ACM Journal of Experimental Algorithmics, Vol. 14, Article No. 4.4, Publication date: September 2009.

actually contained in the set. Since the data structure is randomized by using hash functions, reporting a false-positive occurs with a certain probability, called the false-positive rate (FPR). This impreciseness also makes it impossible to remove an element from a Bloom filter. The advantage of a Bloom filter over the established dictionary structures is space efficiency. A Bloom filter needs only a constant number of bits per (prospective) element, while keeping the FPR constant, independent of the size of the elements' universe.

The false positives can often be compensated for by recalculating or retransmitting data. Bloom filters have applications in the fields of databases, network applications [Broder and Mitzenmacher 2004], and model checking [Dillinger and Manolios 2004a, 2004b]. The requirements on the Bloom filter and the way of using it differ greatly among these fields of applications.

1.1 Article Outline

In Section 2, we review “standard” Bloom filters, that are based on setting k bits in a bit array, which are determined by k -hash functions. Section 3 introduces and analyzes a family of cache-efficient variants of standard Bloom filters. There are two main ideas here: concentrate the k bits in one (or only few) cache blocks and precompute random bit patterns in order to save both hash bits and access time. While these Bloom filter variants improve execution time at the cost of slightly increased FPR, the filters presented in Section 4 save space by engineering practical variants of the theoretically spaced optimal Bloom filter replacements proposed by Pagh et al. [2005]. The basic idea is a compressed representation of a Bloom filter with $k = 1$. Our main algorithmic contribution is the observation that a technique from information retrieval fits perfectly here: Since the distances between set bits are geometrically distributed, Golomb codes yield almost optimal space [Moffat and Turpin 2002]. After giving some hints on the implementation in Section 5, we present an experimental evaluation in Section 6. We conclude our article in Section 7.

2. STANDARD BLOOM FILTERS WITH VARIANTS

The original Bloom filter for representing a set of at most n elements consists of a bit vector of length m . Let $c := m/n$ be the bits-per-element rate. Initially, all bits are set to 0. To insert an element e into the filter, k bits are set in the Bloom filter according to the evaluations of k independent hash functions $h_1(e), \dots, h_k(e)$. The membership query consists of testing all those bits for the query element. If all bits are set, the element is likely to be contained in the set, otherwise, it is surely not contained.

For a fixed number of contained elements and fixed space, the FPR is lowest possible if the probability of a bit being set is as close as possible to $\frac{1}{2}$. One can easily show that it is optimal to choose the integer k close to $\ln 2 \cdot c = \ln 2 \cdot \frac{m}{n} \approx 0.693 \frac{m}{n}$ (try out whether rounding $\ln 2 \cdot c$ up or down is better).

The probability that a bit has remained 0 after inserting n elements, is¹

$$p' := \left(1 - \frac{1}{m}\right)^{kn} \stackrel{i=kn}{\approx} \lim_{i \rightarrow \infty} \left(1 - \frac{kn}{mi}\right)^i = e^{-kn/m}. \quad (1)$$

The false-positive rate for a standard Bloom filter (std) is

$$f_{\text{std}}(m, n, k) = (1 - p')^k = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k \approx (1 - e^{-kn/m})^k \stackrel{!}{\approx} \frac{1}{2^k} \quad (2)$$

for the optimal k . The detailed calculation can be found in Broder and Mitzenmacher's survey paper [2004].

2.1 Classification

The original Bloom filter can be termed a semidynamic data structure, since it does not support deletions.² Variants that do support deletion are called dynamic. The other extreme is a static filter where not even additions to the set may occur after construction.

2.2 Existing Variants for Different Requirements

A variant called counting Bloom filters [Fan et al. 2000] allows deletion of elements from the Bloom filter by using (small) counters instead of a single bit at every position. This basic technique is also used by Pagh et al. [2005] in combination with a space-efficient multiset data structure to yield an asymptotically space-optimal data structure.

2.3 More Related Work

In Kirsch and Mitzenmacher [2006], it is shown that the prerequisite of independent hash functions can be weakened. The hash values can also be computed from a linear combination of two hash functions $h_1(e)$ and $h_2(e)$. This trick does not worsen the false-positive rates in practice.

Lumetta and Mitzenmacher [2006] improve the FPR of Bloom filters by choosing between two hash function families. However, this needs more hashing and causes more cache faults, and hence is contrary to our approaches.

Buhrman et al. [2002] consider dictionary data structures that answer queries using a single bit probe and with an error probability of ϵ . However, note that this data structure is quite different from the bitvector-based Bloom filter replacement we consider in Section 4. Our data structure needs much less memory, whereas the one in Buhrman et al. [2002] has the advantage that the error probability can be made arbitrarily small using repeated calls.

¹We assume throughout the article that the hash functions are perfectly random.

²We define a deletion as having the same effect as if the element was never inserted.

3. BLOCKED BLOOM FILTERS

We will now analyze the cache efficiency of a standard Bloom filter, which we assume to be much larger than the cache. For negative queries, only less than two cache misses are generated, on average. This is because each bit is set with probability $q = 1/2$, when choosing the optimal k , and the program will return false as soon as an unset bit is found. This cannot be improved much, since, at most, one cache fault is needed for accessing some randomly specified cell in the data structure. Thus, standard Bloom filters are cache efficient for negative queries.

For positive queries (both false or true) and insertions, however, standard Bloom filters are cache inefficient, since k cache misses are generated.

In this section, we present a generally cache-efficient variant called blocked Bloom filter (blo). It consists of a sequence of b comparatively small standard Bloom filters (Bloom filter blocks), each of which fits into one cache-line. Currently, a common cache line size is 64 bytes = 512 bits. For best performance, those small filters are stored cache-line-aligned. For each potential element, the first hash value selects the Bloom filter block to be used. Additional hash values are then used to set or test bits as usual, but only inside this one block. A blocked Bloom filter therefore only needs one cache miss for every operation. In the setting of an external memory Bloom filter, the idea of blocking was already suggested in Manber and Wu [1994], where the increase of the FPR was found negligible for the test case there ($k = 5$), and no further analysis was done. The blocked Bloom filter scheme differs from the partition schemes mentioned in Kirsch and Mitzenmacher [2006], where each bit is inserted into a different block.

Let primed identifiers refer to the “local” parameters of the Bloom filter block. At first glance, blocked Bloom filters should have the same FPR as standard Bloom filters of the same size since the FPR in Equation (2) only depends on k and n/m , k equals k' , and the expected value of n'/m' is n/m . However, we are dealing with small values of m so that the approximation is not perfect. More importantly, n' is a random variable that fluctuates from block to block. Some blocks will be overloaded and others will be underloaded. The net effect is not clear at first glance. The occupancies of the blocks follow a binomial distribution $B(n, 1/b)$ that can be closely approximated by a Poisson distribution with parameter $n/b = B/c$, since n is usually large, and B/c is a small constant. An advantage of this approximation is that it is independent of the specific value of n . For the overall FPR of a blocked Bloom filter with local FPR $f_{\text{inner}}(B, i, k)$, we get the following infinite but quickly converging sum:

$$f_{\text{blo}}(B, c, k) := \sum_{i=0}^{\infty} \text{Poisson}_{B/c}(i) \cdot f_{\text{inner}}(B, i, k). \quad (3)$$

For a blocked Bloom filter using the typical value $c = 8$ bits per element, the decline in accuracy is not particularly bad; the FPR is 0.0231 instead of 0.0215 for $B = 512$ bits. By increasing c by 1, we can (over-)compensate for that. For larger c , the effect of the nonuniform distribution can be diminished by choosing a smaller k than otherwise optimal. Still, for $c = 20$ and $k = 14$, the FPR almost triples: It rises from 0.0000671 to 0.000194, which might not

Table I. Increasing the Space for a Blocked Bloom Filter to Compensate the FPR ($B = 512$)

c	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
c'	6	7	8	9	10	11	12	13	14	16	17	18	20	21	23
+%	20	16	14	12	11	10	9	8	7	14	13	12	17	16	21
c	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34
c'	25	26	28	30	32	35	38	40	44	48	51	58	64	74	90
+%	25	23	27	30	33	40	46	48	57	65	70	87	100	124	165

be acceptable in certain applications. Thus, we have to increase c to 24. The numerically computed results for many values of c are shown in Table I. These values are impractical for $c > 28$, since more than 50% additional memory must be used to compensate for the blocking. However, for $c < 20$, the additional memory required is only 20%. This can be acceptable, and often even comes with an improvement to the FPR, in the end. For $c > 34$, the blocked Bloom filter with $B = 512$ cannot compensate the FPR any more, for a reasonable number of bits per element.

3.1 Bit Patterns (pat)

A cache miss is usually quite costly in terms of execution time. However, the advantage in performance by saving cache misses can still be eaten up if the computation is too complex. For the blocked Bloom filters, we still have to set or test k bits for every insertion or positive query. On the other hand, modern processors handle up to 128 bits in a SIMD instruction. The latest models even have two SIMD units. Hence, a complete cache-line can be handled in only two steps.

To benefit from this functionality, we propose to implement blocked Bloom filters using precomputed bit patterns. Instead of setting k bits through the evaluation of k hash functions, a single hash function chooses a precomputed pattern from a table of random k -bit pattern of width B . In many cases, this table will fit into the cache. With this solution, only one small (in terms of bits) hash value is needed, and the operation can be implemented using few SIMD instructions. When transferring the Bloom filter, the table need not be included explicitly in the data, but can be reconstructed using the seed value.

The main disadvantage of the bit pattern approach is that two elements may cause a table collision when they are hashed to the same pattern. This leads to an increased FPR. If ℓ is the number of table entries, the collision probability in an n element Bloom filter block is $p_{\text{coll}}(n, \ell) := 1 - (1 - \frac{1}{\ell})^n$. Hence, we can bound the FPR for one block by

$$f_{\text{pat}}(m, n, k, \ell) \leq p_{\text{coll}}(\ell) + (1 - p_{\text{coll}}(\ell))f_{\text{std}}(m, n, k). \quad (4)$$

This local FPR can be plugged into Equation (3) to yield the total FPR. Bit patterns work well when on the one hand, the pattern table is small enough to fit into the cache, and on the other hand, the table is big enough to ensure that table collisions do not increase the FPR by too much.

3.2 Multiplexing Patterns

To refine this idea once more, we can achieve a larger variety of patterns from a single table by bitwise-or-ing x patterns with an average number of k/x set bits. Ignoring rounding problems, dependencies, and so on

$$f_{\text{pat}[x]}(m, n, k, \ell) \approx f_{\text{pat}}(m, xn, k/x, \ell)^x. \quad (5)$$

3.3 Multiblocking

One more variant that helps improving the FPR is called multiblocking. We allow the query operation to access X Bloom filters blocks, setting or testing k/X bits, respectively, in each block. (When k is not divisible by X , we set an extra bit in the first $k \bmod X$ blocks.) Multiblocking performs better than just increasing the block size to XB , since more variety is introduced this way. If we divide the set bits among several blocks, the expected number of 1 bit per block remains the same. However, only k/X bits are considered in each participating block, when accessing an element. Thus, we have to generalize Eq. (2):

$$f_{\text{std}[X]}(m, n, k) = \left(1 - \left(1 - \frac{1}{m}\right)^{kn/X}\right)^k. \quad (6)$$

We get an estimate for the total FPR of

$$f_{\text{blo}[X]}(B, c, k) := \sum_{i=0}^{\infty} \text{Poisson}_{XB/c}(i) \cdot f_{\text{std}[X]}(B, i/X, k)^X. \quad (7)$$

This can be adapted to bit patterns as before. The multiplexing and the multiblocking factor will be denoted by appending them to either variant, that is, $\text{blo}[X]$ and $\text{pat}[x, X]$, respectively.

3.4 Combinations

Using the formulas presented, all combinations of the variants presented here can be theoretically analyzed. Although the calculations make simplifying assumptions, mainly through disregarding dependencies, they match the experimental results closely, as shown in Figure 1. The differences are very small, and only appear for large c , where random noise in the experimental values comes into play.

Hash Complexity

Besides the cost for memory access, Bloom filters incur a cost for evaluating hash functions. Since the time needed for this is very application dependent, we choose to compare the different algorithms based on the number of hash bits needed for a filter access. Table II summarizes the results (the *ch* and *gcs* methods are discussed in Section 4).

Exemplary values for $m=800,000,000$ are shown in Figure 2. The values follow the theoretical computation in Table II. Obviously, the proposed variants perform better than the standard Bloom filters, for a reasonable choice of c .

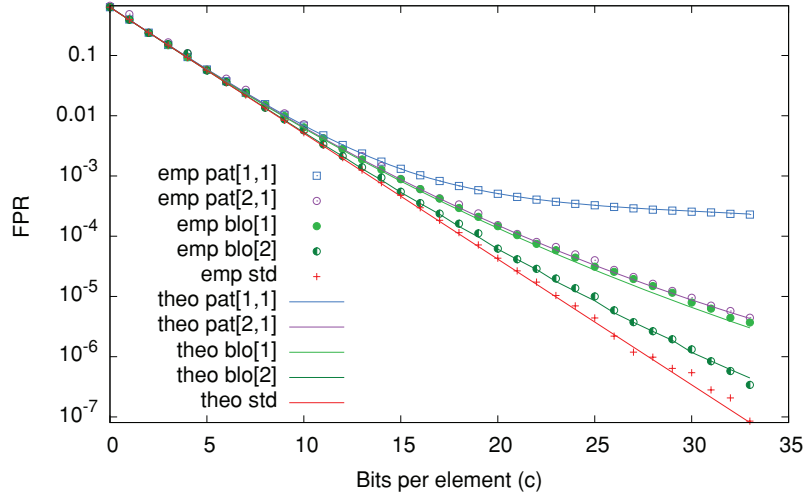


Fig. 1. Comparing the empirical FPR to the theoretically computed one, for some variants. The lines represent the theoretical, and the points indicate the experimental results.

Table II. Number of Hash Bits Used by the Various Variants. f is the Desired FPR, n the Number of Elements, and m the Available Space; B the Block Size, and ℓ the Length of the Pattern Table. Throughout This Article, $\log x$ Stands for $\log_2 x$

Operation	Insert/Positive Query	Negative Query
std	$k \log m$	$2 \log m$ (expected)
blo[X]	$X \log(m/B) + k \log B$	$\log(m/B) + 2 \log B$
pat[x, X]	$X(\log(m/B) + x \log \ell)$	$\log(m/B) + x \log \ell$
ch	$\log n/f$	$\log n/f$
gcs	$\log n/f$	$\log n/f$

4. SPACE-EFFICIENT BLOOM FILTER REPLACEMENTS

In the previous section, we proposed methods for making Bloom filters produce less cache faults and use less hash bits. The aim was to improve the execution time, while at the same time, sacrificing FPR and/or space efficiency. In this section, we describe Bloom filters with near-optimal space consumption that are also cache-efficient. We pay for this with a trade-off between execution time and an additive term in the space requirement.

Our basic solution is also static, that is, the data structure must be constructed in a preprocessing step, with all elements given, before any query can be posed. At the end of this section, we outline how the data structure can be dynamized.

The original Bloom filters are space-efficient, but not space-optimal [Pagh et al. 2005]. When ignoring membership query time, one could just store one hash value in the range $\{1, \dots, n/f\}$ to obtain an FPR of f . This would cost only $\log \binom{n/f}{n} \approx n \log \frac{e}{f}$ bits instead of $\frac{n}{\ln 2} \log(1/f)$ bits. Hence, a traditional Bloom filter needs about $1/\ln 2 \approx 1.44$ times as much space, even worse by an additive

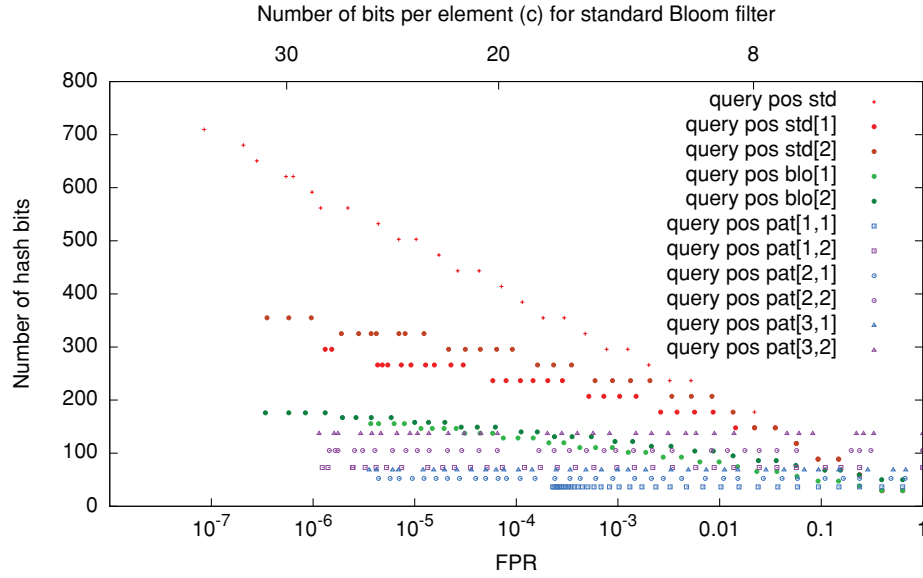


Fig. 2. Number of hash bits used against the FPR.

constant when compared to the information-theoretic minimum, $n \log(1/f)$ bits. This amount of extra memory can be saved by sacrificing some access time. Pagh et al. [2005] use a asymptotically space-optimal hash data structure invented by Cleary [1984] for storing just those hash values. Let this approach be termed CH filter (ch) here. However, to guarantee expected constant membership query time, a constant number of bits must be spent additionally for each contained element. Those bits comprise a structure that gives hints to find the desired element quickly. The more bits are provided, the faster the data structures will work. Although the number of bits is independent of n , and more importantly, of the FPR, this eats up most of the savings achieved, for reasonably small values of c . Another point is that a hash data structure should never get close to full, that is, there must be some maximal load α , which in turn increases memory usage. Summarizing this, access time must be traded off with space efficiency again, but this time with the ability to get arbitrarily close to the theoretical optimum, asymptotically.

Our own solution proposed here is based on an approach used in search engines to store sorted lists of integers [Sanders and Transier 2007]. Imagine a simple Bloom filter with $k = 1$, that is, a hashed bitmap, yielding an FPR of $1/c$. This bitmap can be greatly compressed, as the 1 bits are sparse.³ We do not directly compress the bitmap, but the sorted sequence of hash values in the range $\{0, \dots, nc\}$ for all the contained elements. These values are uniformly distributed, therefore, the differences between two successive values are (approx-

³In Mitzenmacher [2001], this is exploited using arithmetic coding. Our approach is related but more directly uses the structure of the problem and directly leads to an approach that allows random access.

imately) geometrically distributed with $p = 1/c$. For a geometric distribution, Golomb coding [Moffat and Turpin 2002, p. 36] is a minimal-redundancy code, that is, by choosing the right parameter, we lose only at most half a bit per element, compared to the information-theoretic optimum.

However, this compressed sequence still does not allow any random access, since the Golomb codes have value-dependent sizes. Hence, we have to augment it with an index data structure so we can seek to a place near the desired location quickly. Therefore, we divide the number range of the hash function into parts of equal size I . In addition, for each of these blocks, a bit-accurate pointer to the beginning of the subsequence that contains the corresponding values, is stored. So there is a trade-off once again: For a small search time, we want a small I , but large I are good for saving space.

This data structure, termed golomb-compressed sequence (gcs) is static, in contrast to the compact-hash approach, that is, all hash values (and thus all elements in the set) must be known beforehand.

4.1 Dynamization of gcs

We can support insertions by maintaining a small dynamic hash table T_i for recently inserted elements. It suffices if T_i stores the bit positions for the main table. When T_i becomes too big, we empty it by reconstructing the main table. With a bit of caution, we can even support deletion using a deletion buffer T_d . This works if both the main table and T_d store multisets of bit positions. This can be done very space efficiently in the main table. We just need to provide a code word for the distance 0. Since this does not significantly increase the lengths of the other code words and since there are only few collisions, the resulting space and time overhead is small.

5. IMPLEMENTATION ASPECTS

Blocked Bloom filters with bit patterns profit from storing the Bloom filter in negated form—a membership query then reduces to testing whether the bitwise-and of the pattern and the negated filter block is zero. Insertion amounts to a bitwise-and operation of the negated pattern and the negated filter block.

To scale the hash values to the appropriate range, we use floating-point multiplication and rounding instead of division with remainder. Our measurements indicate that this is crucial for performance.

We implemented all algorithms in a library-style way that makes them easy to use in real applications. This includes easy configuration of all tuning parameters, most of them allowed to be changed at runtime. Through generic programming, we are able to plug in arbitrary data types and hash functions, without any runtime overhead. The code can be obtained from the authors.

With all those details described, we can state that everything possible was done to achieve best practical performance for all of the contestants, thus guaranteeing a fair comparison.

Assume we have two bit vectors, the pattern p , and the filter f . We want to tests whether all 1 bits in p are also set in f . All operations in the following

are supposed to be executed bitwise on the vectors.

$$((p \Rightarrow f) = \bar{1}) \Leftrightarrow (p \wedge f = p) \Leftrightarrow ((p \wedge f) \oplus p = \bar{0}) \quad (8)$$

Now, keep the filter in an inverted fashion, always. Then, there is only one binary operation and one comparison against 0 necessary.

$$((p \Rightarrow f) = \bar{1}) \Leftrightarrow (\neg p \vee f = \bar{1}) \Leftrightarrow (p \wedge \neg f = \bar{0}) \quad (9)$$

For the insert operation, we only need one more binary inversion. Overall, since an inversion is cheaper than a binary operation, we use the inverted version.

6. EXPERIMENTAL EVALUATION

We evaluate our implementations using one core of an Intel Xeon 5,140 dual-core processor, running at 2.33GHz with 4MB of L2-cache using 64-Byte cache lines. They use the automatic vectorization feature of the Intel C++ Compiler⁴ to enable SIMD operations without hand-written assembler code. We benchmark the operations

- (1) Insert an element.
- (2) Query an element contained in the set, returning true.
- (3) Query an element not contained in the set, returning true or false.

The elements are strings of length 8. They are hashed using one or two variants of Jenkins' hash function [Dillinger and Manolios 2004b] with different seeds that output a total of 64 bits. When even more bits are needed, hash values are generated as needed using a linear combination of those two hash values, as proposed in Kirsch and Mitzenmacher [2006]. In each case, the number of elements n is chosen so that the filter data structure has size 95MB. After inserting n elements, querying for the same set is performed. Additional n elements are queried for in the second step. This made it possible to measure the running times for both positive and negative queries. The cache-line size is 64 bytes, so we chose $B = 512$. For the pattern-based filters, the table size is set to the full 4MB, resulting in $\ell = 64K$.

To make the comparison fair, we also include variants of the standard Bloom filter that for a given c use a k value just large enough to ensure an FPR at least as good as $\text{blo}[1]$ ($\text{std}[1]$) and $\text{blo}[2]$ ($\text{std}[2]$), respectively.

Figures 3 and 4 show running times for the positive and negative queries, as well as the filter size, for c from 1 to 34. The insertion times are omitted since they are very similar to the times for positive queries.

As stated before, there is not much improvement to expect for negative queries, since their detection is already quite cache-efficient for the original Bloom filter. Also, they do not use many hash bits. For both positive queries and insertions, the blocked Bloom Filter variants outperform the original Bloom filter, in particular for low FPRs, that is, high reliability. The maximum speed-up

⁴Version 9.1.045

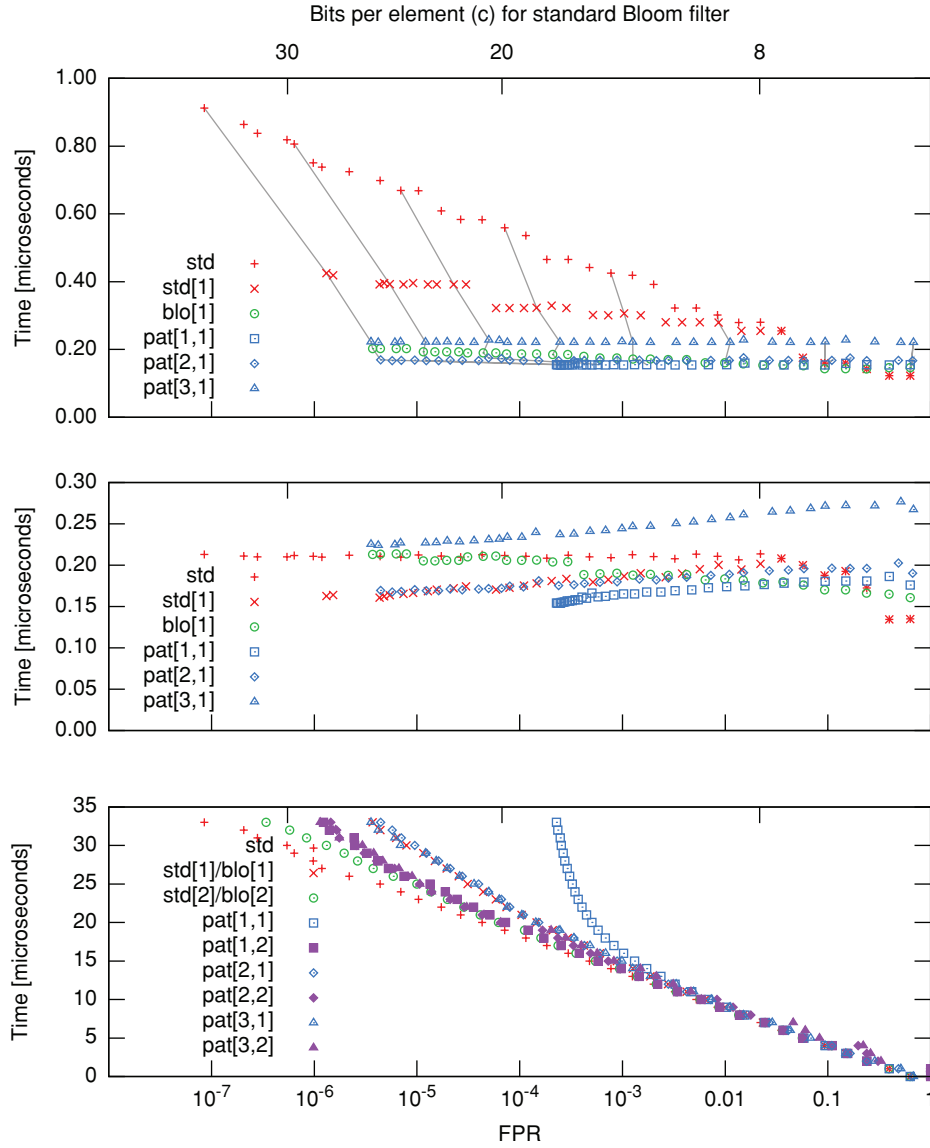


Fig. 3. Execution times for positive (top), negative (middle) queries, and size of filter in bits per contained element (bottom). For comparison, lines connect data points with the same number of bits per element. For readability, only the variants accessing one block are shown here in (top) and (middle), the two-block variants can be found in Figure 4.

factor is close to 4, using 32% more memory than the standard variant. However, the speed-up is actually smaller than one would expect from the difference in cache misses (see Appendix A). Apparently, the system can hide cache latencies using prefetching. A system based on a different architecture shows similar results (see Appendix B).

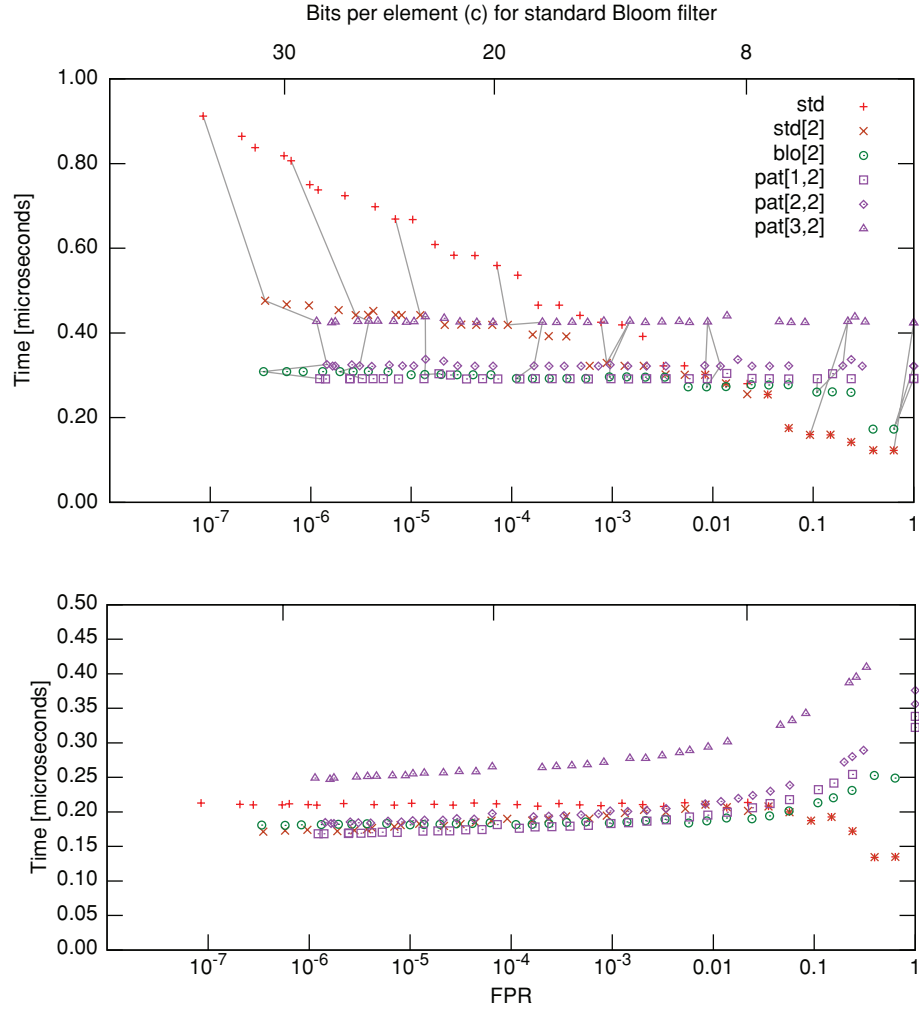


Fig. 4. Execution times for positive (top) and negative (bottom) queries for the variants accessing two blocks.

The normal pattern variants are only slightly faster than the regular blocked Bloom filter. One reason is that we use very cheap hash functions, but there is another cause: When the pattern table occupies all of the cache, almost every filter access evicts a pattern from the cache whose next access will cause a cache fault. Reducing the table size (slowly) reduces this problem. The normal pattern variant also does not reach the area of very large FPRs. For $c = 34$, the FPR is limited by the probability of table collisions, about $512/34/64K \approx 2.310^{-4}$. The multiplexing versions are able to overcome this limitation, but need more computational effort.

Regarding the single-blocking variants, for positives and insertions, pat[1, 1] performs best for rather high FPRs, while pat[2, 1] extends this behavior to smaller FPRs, using twofold multiplexing.

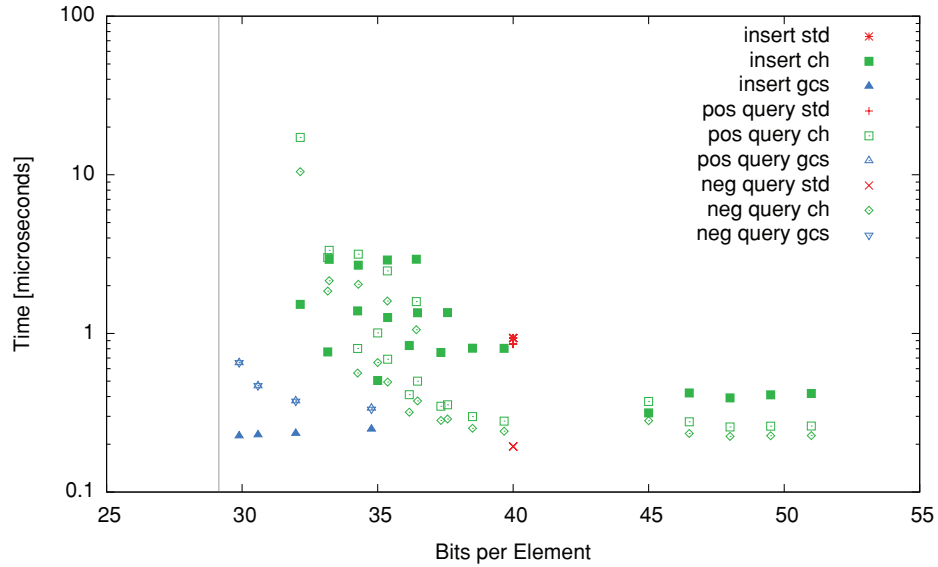


Fig. 5. Execution times of the different operations for many variants and tuning parameters, FPR equivalent to a standard Bloom filter with $c = 40$ and optimal k .

Regarding the two-blocking variants, for positives and insertions, `pat[1, 2]` performs best, being up to almost twice as fast as any standard variant, for a low FPRs. In this range, `pat[1, 2]` can also compete for negative queries. When an even smaller FPR is requested, `blo[2]` should be used, which is only marginally slower than `pat[1, 2]`.

We performed a similar test for the space-efficient replacements. The `ch` data structure used 3 additional bits per entry, while varying the load factor from 0.90 to 0.99. The results are stated in Figure 5, comparing to the standard Bloom filter for $c = 40$, all featuring the same FPR. For this FPR, the lower bound in space for storing the hash values is $-\log 4.5110^{-9} + \log e = 29.14$ bits per element. The minimum space requirement in this experiment for `gcs` is in fact 29.14 bits ($I \rightarrow \infty$), reaching the optimum, while for `ch`, it is 30.80 bits ($\alpha \rightarrow 1$ and omitting one redundant helper bit per entry). For `gcs`, the index data structure can be easily and flexibly rebuilt after compact transmission, but for `ch`, the whole filter must be rebuilt to achieve acceptable execution times.

As we can see, the static `gcs` implementation provides excellent performance when the memory limitations are tight. If more space is available, Compact hash (`ch`) gives better query times, but collapses in terms of insertion performance.

7. CONCLUSION

Which variant or replacement of a Bloom filter works best depends a lot on the application. Standard Bloom filters are still a good choice in many cases. They are particularly efficient for negative queries. Even insertions and positive queries work better than one might think because modern hardware can

Table III. Number of Cache Misses (in Millions) for Various Algorithms, Operations, and Table Sizes

Algorithm	Table Size/Operation					
	64KB		2048KB		4096 KB	
	insert/pos	neg	insert/pos	neg	insert/pos	neg
std	539	112	539	112	539	112
blo[1]	39	39	39	39	39	38
blo[2]	77	39	77	39	77	39
pat[1,1]	39	39	55	55	64	64
pat[1,2]	77	41	109	55	128	65
pat[2,1]	39	39	55	55	64	64
pat[2,2]	77	41	109	55	128	65
pat[3,1]	39	39	55	55	64	64
pat[3,2]	77	41	109	55	128	65

mitigate cache faults by overlapping multiple memory accesses and because a reduction of k below the “optimal” value brings considerable speed-up at moderate increase in space consumption or FPR. Blocked Bloom filters, possibly together with precomputed bit patterns, can mean a significant speedup if insertions and positive queries are important operations or when hash bits are expensive. Multiplexing and multiblocking Bloom filters become important when a very low FPR is required. Space-efficient Bloom filter replacements are particularly interesting when one wants to reduce communication volume for transferring the filter. Somewhat surprisingly, the price one pays in terms of access time is small or even negative if one uses our implementation based on bucketed Golomb coding. Recently, Patrascu [2008] has developed a sophisticated approach that gives a much better asymptotic trade-off between access time and space overhead for data structures, such as our bucketed Golomb coding. It might be interesting to see whether such data structures give good performance in practice.

If internal space efficiency is less important than access time and saving communication volume, one could accelerate our implementation further by using Golomb coding only for the communication and by using a faster representation internally.

We believe that, independent of the particular results, our article is also instructive as a case study in algorithm engineering and its methodology: Modeling both the machine (cache, prefetching, SIMD instructions) and the application (operation mix, difficulty of hashing) are very important here. The article contains nontrivial components with respect to design, analysis, implementation, experimental evaluation, and algorithm library design. In particular, the analysis is of a “nonstandard” type, that is, analysis only seems tractable with simplifying assumptions that are then validated experimentally.

APPENDIX

A. CACHE-EFFICIENCY EVALUATION

Table III lists the number of cache faults that are triggered by executing the experiment. We used a table of size either 64KB, 2048KB, or 4096KB. For $c = 20$, we inserted and queried 40,000,000 elements, respectively. blo[1] causes about

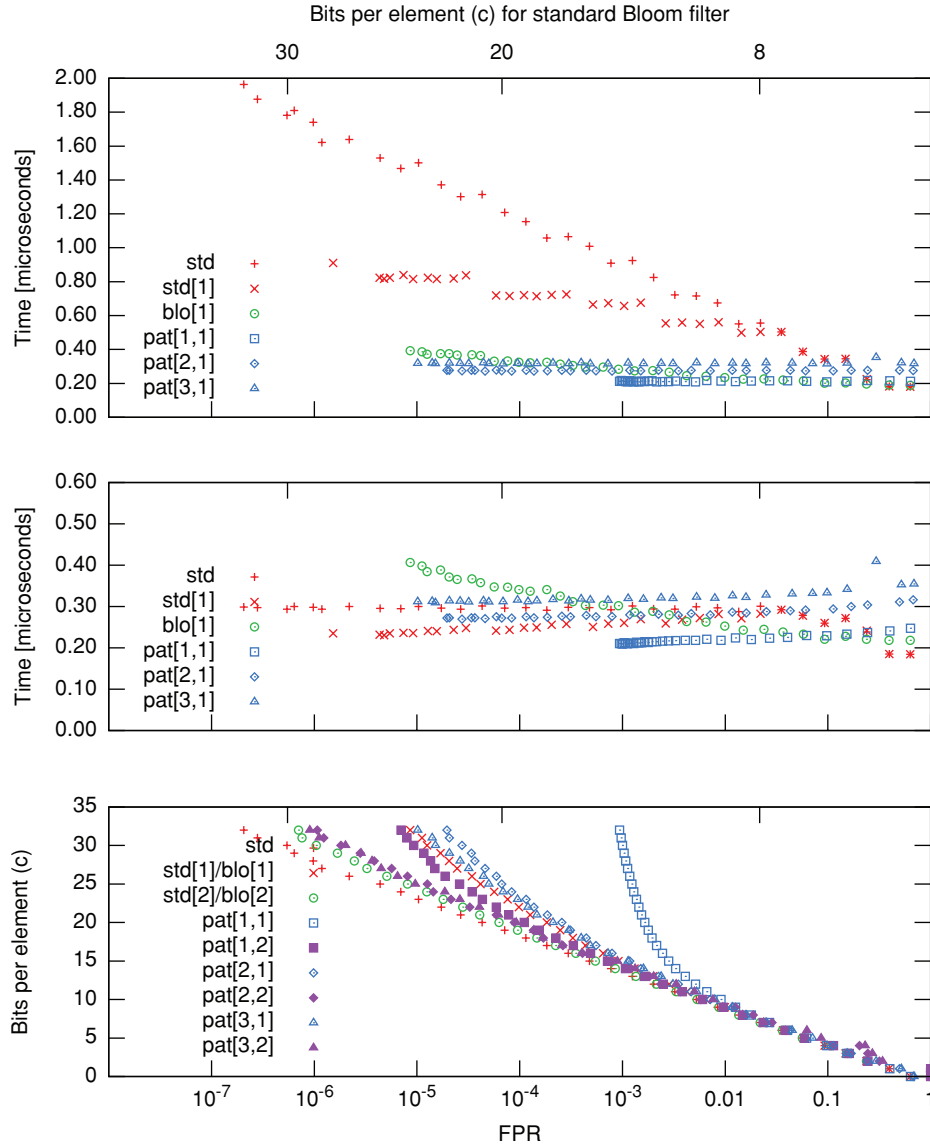


Fig. 6. Execution times for positive (top), negative (middle) queries, and size of filter in bits per contained element (bottom). For readability, only the variants accessing one block are shown here in (top) and (middle), the two-block variants can be found in Figure 7.

one cache miss per operation, which is quite accurately reflected by the numbers. For a insertion or positively answered query, the number of cache faults is reduced by a factor of 13.96 compared to std. This is also just as expected, since $k = 14$. However, Figure 3 indicates that for $c = 20$, blo[1] is only about three times faster than std. Part of the explanation is that the number of hash bits needed by the two schemes only differs by a factor of about 3. However,

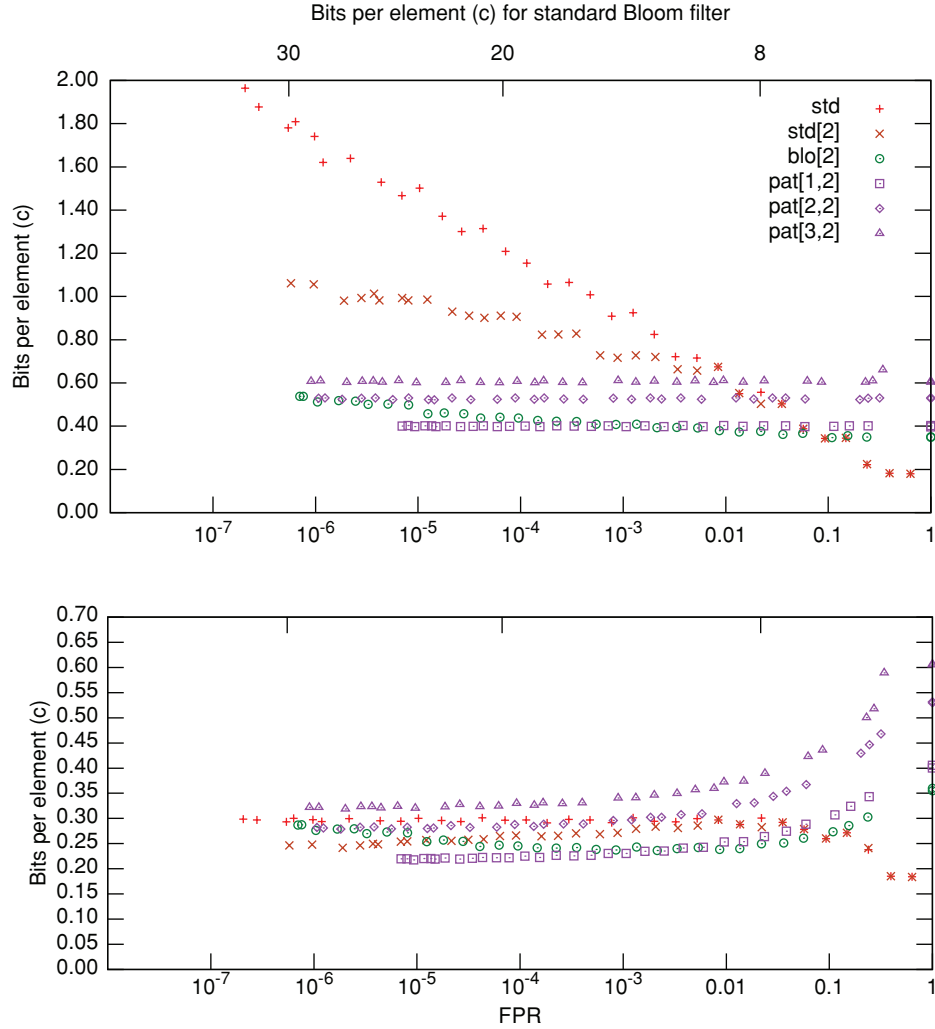


Fig. 7. Execution times for positive (top) and negative (bottom) queries for the variants accessing two blocks.

since the execution time is still dominated by the memory access times, an important part of the explanation seems to be that the compiler schedules memory accesses (or prefetches) already in the loop iteration before the actual use of the data. Thus, cache latency can be hidden behind other operations. This prefetching behavior also explains why there are about 2.9 cache faults per negative query of std although the analysis predicts only two. Apparently, once the query algorithm found a zero bit, one more memory access has already been issued.

For the two-blocking variants, the number of cache misses obviously doubles. When using patterns, the pattern table and the accessed blocks fight for the cache. When the table is as large as the cache, the numbers go up by a

factor of 1.7, compared to a table of negligible size. This surprises, since the table entries should clearly dominate the cache sets. Because of the 8-way set-associative cache organization, we would expect only every eighth table access to fail, instead of the measured 70%. Apparently, the cache does not follow a strict LRU policy. Still, the number of cache misses is far lower than for the standard variants. This fighting for the cache could be avoided by using special compiler instructions to bypass the cache when reading or writing blocks.

B. RESULTS FOR AMD PLATFORM

To ensure that our results are not specific for a particular processor architecture, we have repeated the tests from Figures 3 and 4 on an AMD Opteron 844 machine with 1MB cache, running at 1.8GHz (see Figures 6 and 7). The table size was reduced to 1MB, accordingly.

The speed improvements for this platform are even slightly better, the maximum speed-up is about 5 instead of 4. However, the absolute running times are worse, because of the system's worse overall performance. The pat[1,1] filter does not reach as low FPRs as before, because of the shorter pattern table.

ACKNOWLEDGMENTS

We would like to thank M. Dietzfelbinger for valuable discussions and hints how to analyze the false-positive rate of blocked Bloom filters. Frederik Transier provided a first implementation of Golomb coding.

REFERENCES

- BLOOM, B. H. 1970. Space-time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7.
- BRODER, A. AND MITZENMACHER, M. 2004. Network applications of bloom filters: A survey. *Internet Math.* 1, 4.
- BUHRMAN, H., MILTERSEN, P. B., RADHAKRISHNAN, J., AND VENKATESH, S. 2002. Are bit-vectors optimal? *SIAM J. Comput.* 31, 6, 1723–1744.
- CLEARY, J. G. 1984. Compact hash tables using bidirectional linear probing. *IEEE Trans. Comput.* 33, 9, 828–834.
- DILLINGER, P. C. AND MANOLIOS, P. 2004a. Bloom filters in probabilistic verification. In *Proceedings of the 6th International Formal Methods in Computer-Aided Design (FMCAD'04)*. Springer, Berlin, 367–381.
- DILLINGER, P. C. AND MANOLIOS, P. 2004b. Fast and accurate bit-state verification for SPIN. In *Proceedings of the 11th International SPIN Workshop*. Springer, Berlin, 57–75.
- FAN, L., CAO, P., ALMEIDA, J. M., AND BRODER, A. Z. 2000. Summary cache: A scalable wide-area web cache sharing protocol. *IEEE/ACM TON* 8, 3, 281–293.
- KIRSCH, A. AND MITZENMACHER, M. 2006. Less hashing, same performance: Building a better Bloom filter. In *Proceedings of the 14th European Symposium (ESA'06)*. Springer, Berlin, 456–467.
- LUMETTA, S. AND MITZENMACHER, M. 2006. Using the power of two choices to improve bloom filters. <http://www.eecs.harvard.edu/~michaelm/postscripts/bftwo.ps>.
- MANBER, U. AND WU, S. 1994. An algorithm for approximate membership checking with application to password security. *Inform. Process. Lett.* 50, 4 (25), 191–197.
- MITZENMACHER, M. 2001. Compressed Bloom filters. In *PODC 2001*. 144–150.
- MOFFAT, A. AND TURPIN, A. 2002. *Compression and Coding Algorithms*. Kluwer.
- PAGH, A., PAGH, R., AND RAO, S. S. 2005. An optimal Bloom filter replacement. In *Proceedings of the 16th Annual Symposium on Discrete Algorithms (SODA'05)*. ACM, New York, 823–829.

- PATRASCU, M. 2008. Succincter. In *Proceedings of the 49th Annual IEEE Symposium on Foundation of Computer Science (FOCS'08)*. IEEE, Los Alamitos, CA, 305–313.
- SANDERS, P. AND TRANSIER, F. 2007. Intersection in integer inverted indices. In *Proceedings of the 9th Workshop on Algorithm Engineering and Experiments (ALENEX)*. Society for Industrial and Applied Mathematics, Philadelphia.

Received January 2009; accepted July 2009